OBJECT ORIENTED PROGRAMMING

Python is an interpreted, high level, general purpose programming language.

Programming paradigms:

Programming paradigms are the way to classify programming languages based on their features.

Languages can be classified into multiple paradigms.

Common programming paradigms are:

1) Scripting

2) Structured programming

3) Modular programming

4) Object oriented programming

Python supports scripting, structured programming, modular programming & object oriented programming.

Scripting:

In a scripting language, script is a sequence of commands written as a text and run by an interpreter.

Structured Programming:

In structured programming, we divide the program into set of functions, in which each function works as a sub program.

Modular Programming:

Modular programming is the process of dividing a program into separate sub programs. A module is a separate software component.

Object Oriented Programming:

Object oriented programming allows constructing a program using a set of objects and their interactions.

A better style of programming is called object oriented programming in which data and functions are combined to form a class.

Classes & Objects are the two main aspects of object oriented programming.

Object Oriented Programming Language(OOPL):

A language that supports all the principles of an object oriented programming is known as an object oriented programming language.

Object Oriented Principles:

1) Encapsulation

2) Abstraction

3) Inheritance

4) Polymorphism

Object Oriented = Object Based + Inheritance + Runtime Polymorphism

Object Based Languages:

Java Script, VB Script, Visual Basic, .. etc.,

Object Oriented Programming Languages:

C++, Java, Python, C#, VC++, Simula, Small talk, Ada, ..etc.,

To use the above principles in a python we required the following language constructs:

1) Class

2) Object

Class:

A class is a collection of variables & methods.

The data can be represented in a class by using variables.

The operations can be represented in a class by using methods.

The syntax of a class:

class ClassName:

    ========

    ========

    ========

Object:

An object is an instance of a class. The concept of allocating memory for members of a class at run time is known as an object in a python.

The syntax to create an object:

object_reference=ClassName()

After creating an object, python interpreter internally takes the original address of an object and mapped to another indirect address and assigned to object reference.

Encapsulation:

The concept of binding variables & methods into a class is known as an encapsulation.

Variables:

A variable is a container that contains the data.

There are four types of variables in python:

1) Instance Variables

2) Class Variables

3) Local Variables

4) Global Variables

Methods:

A group of statement into a single logical unit is called as function.

A function that is defined inside a class is known as method.

There are four types of methods in a python:

1) Instance Methods

2) Class Methods

3) Static Methods

4) Special Methods

Instance Methods:

A method that is defined in a class with self-parameter is called as an instance method.

The self-argument refers to the object itself.

Instance methods can be accessed by using object or object reference.

Examples:

```
class A:
    def show(self):
        print("Welcome")
A().show()
a=A()
a.show()
```

Local Variables:

A variable that is defined inside a method body is known as local variable.

Method parameters are also called local variables.

Local variables memory allocated whenever method is called.

Local variables can be accessed with in the method only.

Local variables cannot be accessed outside the method.

Local variables are used to perform the task.

Local variables can be accessed directly in that particular method only.

Example:

```
class A:
   def show(self,x):
      y=20
      print(x)
      print(y)
a=A()
a.show(10)
```

Global Variables:

A variable that is defined outside the class is known as global variable.

Global variables memory allocated whenever module is loaded.

Global variables can be accessed anywhere in the program.

Global variables can be accessed outside the program also by using module name.

Global variables can be accessed directly.

Example:

```
x=10
class A:
   def show(self):
      print(x)
a=A()
a.show()
print(x)
```